

Using Declarations in C# 8 with Examples

In this article, I am going to discuss **Using Declarations in C# 8** with Examples. Please read our previous article where we discussed [Pattern Matching Enhancements in C#](#) with Examples. In order to understand the using declarations in C#, first, it is important to understand why they are needed.

Why do we need to use “Using Declarations” in C#?

In C#, as a programmer, we use the using statement for declaring disposable variables (that means creating an instance of a class that implements the IDisposable interface) such as File I/O, Databases, Web Services, etc. The using declaration will ensure that the classes that implement the IDisposable interface will call their Dispose method to dispose of the object.

The only problem is that adding a using statement to our code block introduces a new scope block. C# 8.0 using declarations eliminate this problem. It also guarantees that the Dispose method of the IDisposable interface will be called, even if the code throws an exception.

By using the “using” keyword we can declare a variable that tells the compiler that the variable is declared should be disposed of at the end of the enclosing scope. If this is not clear at the moment then don't worry, we will try to understand this concept with examples.

What is the IDisposable Interface in C#?

If you want to release the resources from an object, then you need to use the IDisposable Interface in C#. So, what you need to do is, your class should inherit from the IDisposable interface and should implement the Dispose method. This Dispose method provides a mechanism to release unmanaged resources. The following is the syntax for IDisposable.

```
class Resource : IDisposable
{
    public Resource()
    {
        //Do something
    }

    public void Dispose()
    {
        //Dispose resource
    }
}
```

Using Statement (Old Way) in C#

The using statement can be used to reference a variable or the result from a method that returns an instance, and at the end of the scope defined by the using statement, the Dispose method gets invoked automatically. For a better understanding, please have a look at the below example. In the below example, first, the Resource object will be created within the using statement, then the resource will be used as part of the using block, and when we go out of the using scope or block, the Dispose method of the Resource Object will be automatically called to release the unmanaged resources and the resource will be disposed of by garbage collector. The following example code is self-explained, so please go through the comment lines for a better understanding.

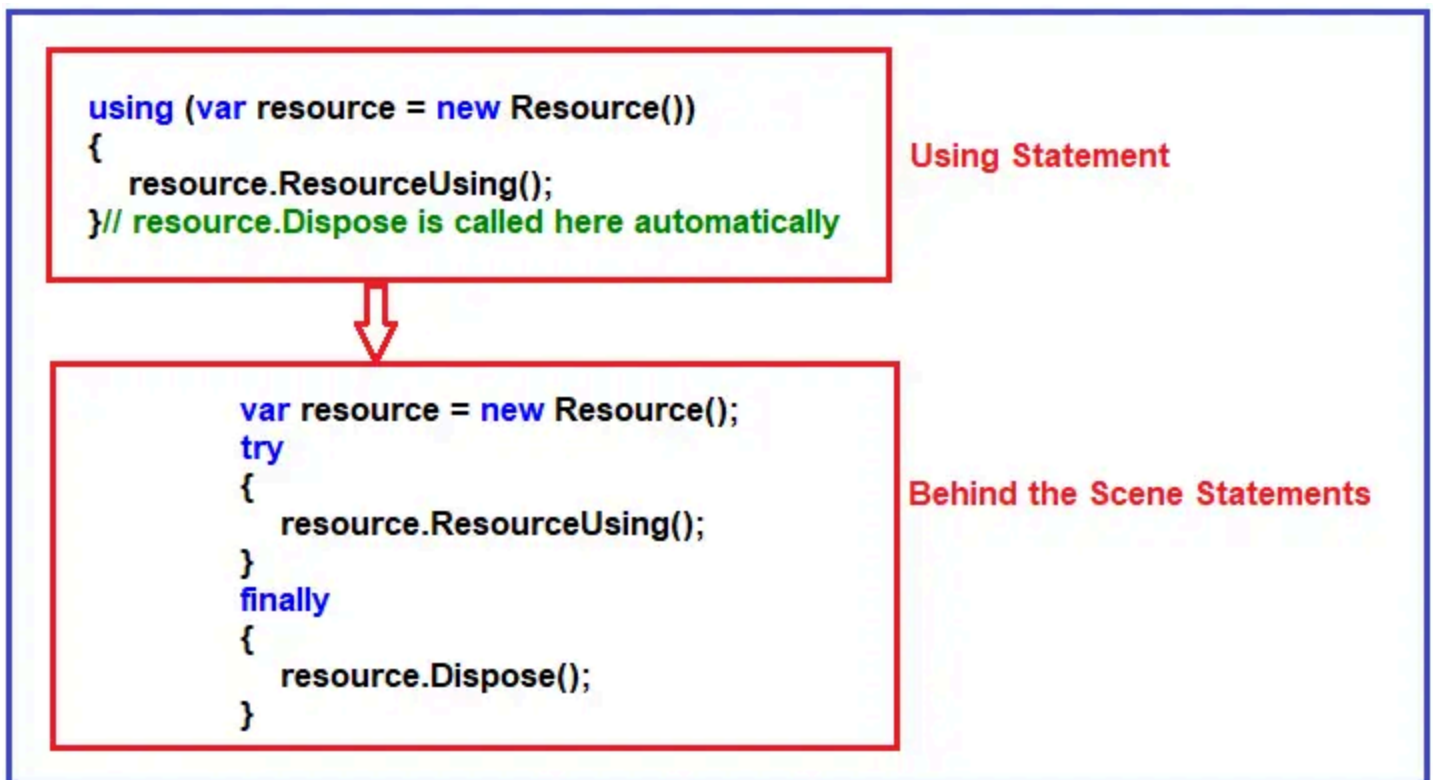
```
using System;
namespace UsingDeclarationsDemo
{
    public class Program
    {
        public static void Main()
        {
            //Creating an Instance of Resource Class using the using declarati
            using (var resource = new Resource())
            {
                //Using the resource as part of the using block
                resource.ResourceUsing();
            } // resource.Dispose Method is called here automatically
            Console.WriteLine("Doing Some Other Task...");
            Console.ReadKey();
        }
    }
    //Creating a Class Inheriting from IDisposable Interface
    class Resource : IDisposable
    {
        public Resource()
        {
            Console.WriteLine("Resource Created...");
        }
        public void ResourceUsing()
        {
            Console.WriteLine("Resource Using...");
        }
        //Providing Implementation for Dispose of IDisposable Interface
        public void Dispose()
        {
            //Dispose resource
            //Performs application-defined tasks associated with freeing, rele
            //resetting
            //unmanaged resources
            Console.WriteLine("Resource Disposed...");
        }
    }
}
```

Output:

```
Resource Created...
Resource Using...
Resource Disposed...
Doing Some Other Task...
```

How is the Dispose Method Automatically Called in C#?

When we use the using statement in C# for creating any object, behind the scenes, the compiler will create a code block using **try/finally** to make sure Dispose method is also called even though an exception is thrown. This is because the finally block gives you the guarantee to be executed irrespective of whether the exception is thrown in the try block or not. So, for a better understanding, please have a look at the below image which shows our using statement code and the compiler-generated code.



Using Declarations in C# 8

With the new C# 8 “using declarations”, the code with the using statement can be simplified. Now, the curly brackets are no longer required. At the end of the scope of the method (which is here at the end of the Main method), the Dispose method is also going to be called automatically. Here also, the compiler creates a try/finally block to make sure Dispose method is called. For a better understanding, please have a look at the below example.

```
using System;
namespace UsingDeclarationsDemo
{
    public class Program
    {
        public static void Main()
        {
```

```

        //Creating an Instance of Resource Class with the new using declar
        using var resource = new Resource();
        //Using the resource
        resource.ResourceUsing();
        Console.WriteLine("Doing Some Other Task...");
    } //Here, the resource.Dispose() Method is called automatically
}
//Creating a Class Inheriting from IDisposable Interface
class Resource : IDisposable
{
    public Resource()
    {
        Console.WriteLine("Resource Created...");
    }
    public void ResourceUsing()
    {
        Console.WriteLine("Resource Using...");
    }
    //Providing Implementation for Dispose of IDisposable Interface
    public void Dispose()
    {
        //Dispose resource
        //Performs application-defined tasks associated with freeing, rele
resetting
        //unmanaged resources
        Console.WriteLine("Resource Disposed...");
    }
}
}

```

Output:

```

Resource Created...
Resource Using...
Doing Some Other Task...
Resource Disposed...

```

Disposing of Multiple Resources in C#:

When we use multiple resources, we are basically used to writing code that looks like the one below with multiple nested using statements.

```

using (var resource1 = new Resource())
{
    using (var resource2 = new Resource())
    {
        resource1.ResourceUsing();
        resource2.ResourceUsing();
    }
}

```

Example to Understand Disposing of Multiple Resources using Statement in C#:

Let us see an example to understand how to dispose of multiple resources with using statements. For a better understanding, please have a look at the following code. Here, first, we are creating resource1 with the using declarations, and then within the body of the first using block, we are creating the second resource object i.e. resource2 with the using declarations. In this case, the first resource2 will be disposed and then resource1 will be disposed of. The following example code is self-explained, so please go through the comment lines for a better understanding.

```

using System;
namespace UsingDeclarationsDemo
{
    public class Program
    {
        public static void Main()
        {
            using (var resource1 = new Resource())
            {
                using (var resource2 = new Resource())
                {
                    resource1.ResourceUsing();
                    resource2.ResourceUsing();
                } //Here, the resource2.Dispose() Method is called automatically
            } //Here, the resource1.Dispose() Method is called automatically
            Console.WriteLine("Main Method End...");
        }
    }
}
//Creating a Class Inheriting from IDisposable Interface
class Resource : IDisposable
{
    public Resource()
    {
        Console.WriteLine("Resource Created...");
    }
    public void ResourceUsing()
    {
        Console.WriteLine("Resource Using...");
    }
}
//Providing Implementation for Dispose of IDisposable Interface
public void Dispose()

```

```

    {
        //Dispose resource
        //Performs application-defined tasks associated with freeing, releasing, or resetting
        //unmanaged resources
        Console.WriteLine("Resource Disposed...");
    }
}

```

Output:

```

Resource Created...
Resource Created...
Resource Using...
Resource Using...
Resource Disposed...
Resource Disposed...
Main Method End...

```

Now, let's do the same with the new using declarations in C# 8. The following code is shorter compared to the previous one. no matter how many resources you need to dispose of.

```

using var resource1 = new Resource();
using var resource2 = new Resource();
resource1.ResourceUsing();
resource2.ResourceUsing();

```

Example to understand Disposing of Multiple Resources using Declarations in C# 8:

In the below example, we are rewriting the previous example using the new using declaration in C# 8.

```

using System;
namespace UsingDeclarationsDemo
{
    public class Program
    {
        public static void Main()
        {
            using var resource1 = new Resource();
            using var resource2 = new Resource();
            resource1.ResourceUsing();
            resource2.ResourceUsing();

            Console.WriteLine("Main Method End...");
        }
    }
}
//Here, resource1.Dispose() and resource2.Dispose() Method is called
automatically

```

```

}
//Creating a Class Inheriting from IDisposable Interface
class Resource : IDisposable
{
    public Resource()
    {
        Console.WriteLine("Resource Created...");
    }
    public void ResourceUsing()
    {
        Console.WriteLine("Resource Using...");
    }
    //Providing Implementation for Dispose of IDisposable Interface
    public void Dispose()
    {
        //Dispose resource
        //Performs application-defined tasks associated with freeing, releasing, or
resetting
        //unmanaged resources
        Console.WriteLine("Resource Disposed...");
    }
}
}

```

Output:

```

Resource Created...
Resource Created...
Resource Using...
Resource Using...
Main Method End...
Resource Disposed...
Resource Disposed...

```

How to Dispose of a Resource Before the Method Ends in C# by Using Declarations?

Let us understand How to Dispose of a Resource Before the Method Ends in C# 8 by Using Declarations with an Example. In this case, we just need to add a separate scope using curly brackets. When the variable is out of scope, the resource is disposed of. For a better understanding, please have a look at the below example. The following example code is self-explained, so please go through the comment lines for a better understanding.

```

using System;
namespace UsingDeclarationsDemo
{
    public class Program
    {
        public static void Main()

```

```

{
    using var resource1 = new Resource();
    resource1.ResourceUsing();
    //Creating a Block using only curly braces
    {
        using var resource2 = new Resource();
        resource2.ResourceUsing();
    } //Here, resource2.Dispose() Method is called automatically
    Console.WriteLine("Main Method End...");
} //Here, resource1.Dispose() Method is called automatically
//Creating a Class Inheriting from IDisposable Interface
class Resource : IDisposable
{
    public Resource()
    {
        Console.WriteLine("Resource Created...");
    }
    public void ResourceUsing()
    {
        Console.WriteLine("Resource Using...");
    }
    //Providing Implementation for Dispose of IDisposable Interface
    public void Dispose()
    {
        //Dispose resource
        //Performs application-defined tasks associated with freeing, releasing, or
resetting
        //unmanaged resources
        Console.WriteLine("Resource Disposed...");
    }
}
}

```

Output:

```

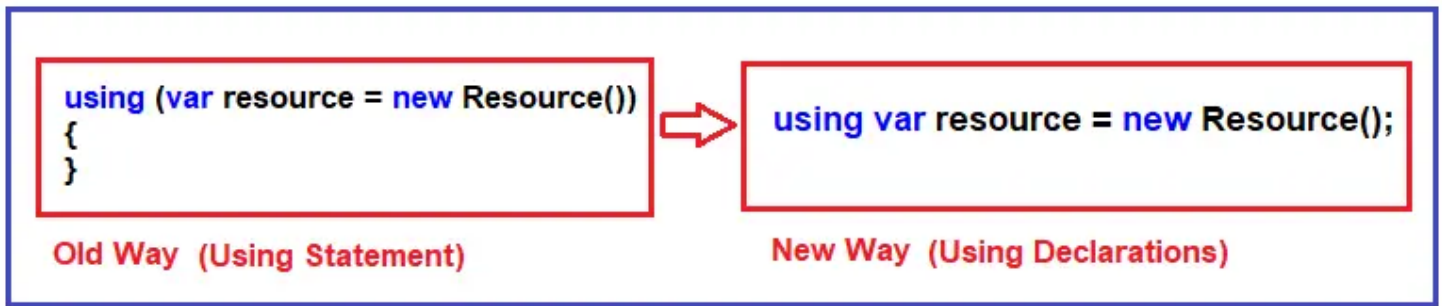
Resource Created...
Resource Using...
Resource Created...
Resource Using...
Resource Disposed...
Main Method End...
Resource Disposed...

```

Using Declarations vs. Using Statements in C#

In the case of using statements in C#, we need to write the try/finally block to ensure that an instance is disposed of in the finally block in case the try code block throws an exception. In the case of many IDisposable types, this would make the code very complex and crowded, as disposing of each instance

requires blocks of try/finally code. On the other hand, using declarations in C# ensure that the object is disposed of when the code leaves the scope it is declared in. Hence, only the main method of a program changes by using declarations rather than using statements.



Realtime Example to Understand the using Declarations in C# 8:

Let's consider the following example in which the file is disposed of when the method ends. Here, we are using the using declarations which was introduced as part of C# 8.

```
using System;
using System.Collections.Generic;
using System.IO;
namespace UsingDeclarationsDemo
{
    public class Program
    {
        public static void Main()
        {
            WriteToFileUsingDeclaration();
            Console.WriteLine("Main Method End...");
        }
        public static void WriteToFileUsingDeclaration()
        {
            List<string> Statements = new List<string>()
            {
                "First Statement",
                "Second Statement",
                "Third Statement."
            };
            using var file = new StreamWriter("D:\\Projects\\MyTextFile.txt");

            foreach (string Statement in Statements)
            {
                file.WriteLine(Statement);
            }
            }// file.Dispose() method call here automatically
        }
    }
}
```

Note: If you go to the definition of StreamWriter class, then somewhere you will find that this class implements the Dispose method of the IDisposable interface. Further notice, this class implements the

TextWriter abstract class and the TextWriter abstract class implements the IDisposable interface.

In the next article, I am going to discuss [Static Local Functions in C# 8](#) with Examples. Here, in this article, I try to explain **Using Declarations in C# 8** with Examples. I hope you enjoy this Using Declarations in C# 8 with Examples article.